DATA ENGINEERING PROJECT

Sales Pipeline Data Warehouse

Airflow → dbt → DuckDB · Dimensional Modelling

A star-schema data warehouse for a CRM sales pipeline. Raw CSV files are loaded by Airflow, transformed by dbt into staging and mart layers, and stored in DuckDB. Full documentation: conceptual, logical and physical model, ETL design, setup guide, tests and data dictionary.

CSV › Airflow › dbt staging › dbt marts › DuckDB

5	4	28	4
source CSV files	mart tables (1 fact + 3 dims)	dbt tests passing	Airflow DAG tasks

PRESENTED BY

Mazen Ashraf

Data Engineering

DOCUMENT DATE

9 October 2026

Contents

1. Overview . . . 3

1.1 What it does . . . 3

1.2 Stack . . . 3

1.3 Project structure . . . 3

1.4 Quick start . . . 4

2. Architecture . . . 5

2.1 Data flow, stage by stage . . . 5

2.2 Layers and schemas . . . 5

2.3 Design decisions . . . 5

3. Dimensional model . . . 6

3.1 Business process and grain . . . 6

3.2 Conceptual schema . . . 6

3.3 Logical schema . . . 6

3.4 Physical schema . . . 7

3.5 Keys . . . 7

4. ETL implementation . . . 8

4.1 Airflow DAG . . . 8

4.2 Extraction script . . . 9

4.3 dbt project . . . 10

4.4 Run results and tests . . . 11

5. Setup and operations . . . 14

5.1 Prerequisites . . . 14

5.2 Run the pipeline . . . 14

5.3 Verify the result . . . 14

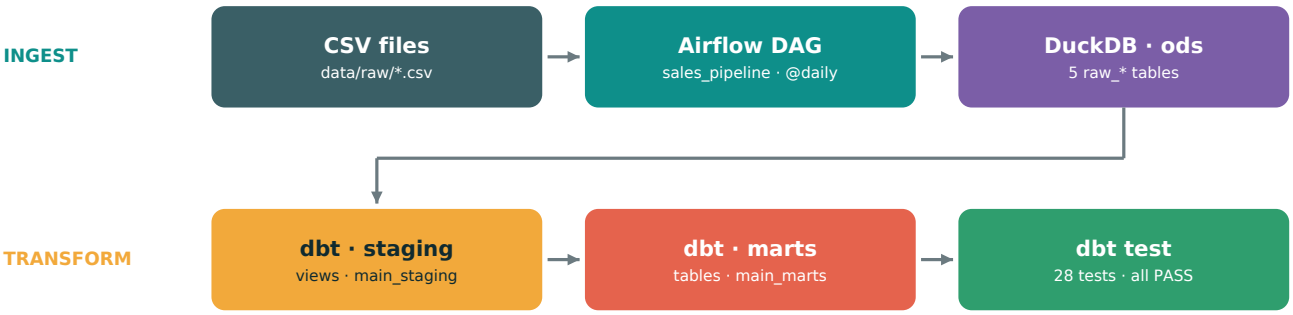
5.4 Troubleshooting . . . 14

6. Limitations and next steps . . . 15

Appendix A. Data dictionary . . . 16

1. Overview

This project builds an analytical data warehouse for a B2B **CRM sales pipeline**. Five CSV files are loaded by **Airflow** into **DuckDB**, transformed by **dbt** into a **star schema** (one fact table and three dimensions) and checked by 28 dbt data tests. The whole pipeline runs from a single Airflow DAG, `sales_pipeline`.



1.1 What it does

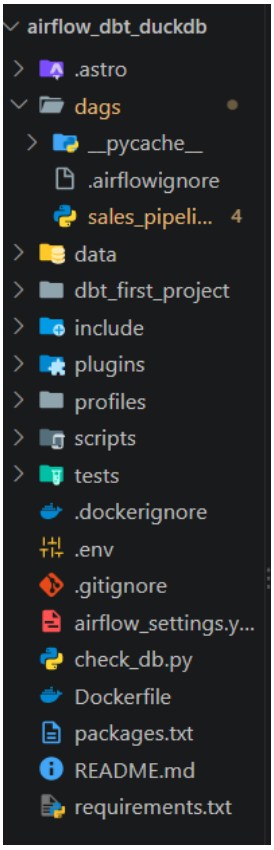
- **Extracts** five CSV files (`accounts`, `products`, `sales_teams`, `sales_pipeline` and the data dictionary) into the `ods` schema with a Python script.
- **Orchestrates** everything with an Airflow DAG: extraction, dbt staging run, dbt marts run, dbt test. It runs daily and retries each task twice.
- **Cleans** the data in dbt staging views (trim text, rename columns, fix the product name `GTXPro`).
- **Models** it in dbt marts: `dim_accounts`, `dim_products`, `dim_sales_teams` and `fact_sales_pipeline`.
- **Validates** it with 28 dbt data tests (not null and unique checks). All of them pass.
- **Documents** the design as a conceptual, a logical and a physical schema (section 3).

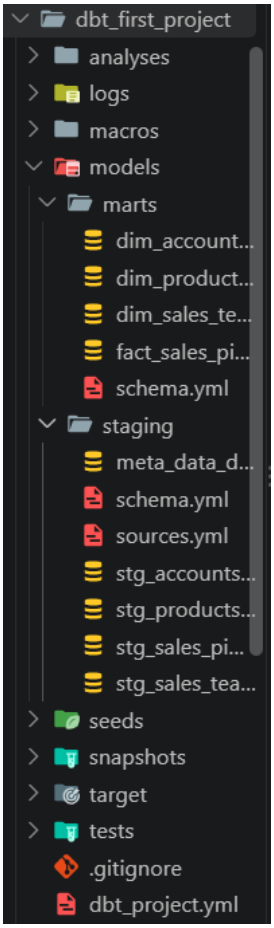
1.2 Stack

Layer	Component	Notes
Orchestration	Apache Airflow on Astro CLI	Astro Runtime in Docker; DAG `sales_pipeline`, schedule `@daily`
Transformation	dbt-core 1.12.5	Adapter `dbt-duckdb` 1.11.0; 9 models, 28 data tests, 5 sources
Warehouse	DuckDB	Single file `include/dev.duckdb` (profile `dev`)
Extraction	Python + DuckDB	`scripts/load_to_ods.py` reads CSV with `read_csv_auto`
Source	CSV files	Kept in `data/raw/`
Modelling	Kimball star schema	Conceptual, logical and physical models

1.3 Project structure

The repository is an Astro project named `airflow_dbt_duckdb`. Airflow files are at the top level, and the dbt project is the folder `dbt_first_project`.





Top level
`dags/` (DAG `sales_pipeline`), `scripts/` (`load_to_ods.py`), `data/raw/` (CSV files), `include/` (DuckDB file), `profiles/` (`profiles.yml`), `Dockerfile`, `packages.txt`, `requirements.txt`, `README.md`.

dbt_first_project/models
`marts/`: `dim_accounts`, `dim_products`, `dim_sales_teams`, `fact_sales_pipeline` and `schema.yml`.

`staging/`: `stg_accounts`, `stg_products`, `stg_sales_pipeline`, `stg_sales_teams`, `meta_data_dictionary`, `sources.yml` and `schema.yml`.

Python packages (`requirements.txt`)
`astronomer-cosmos`, `dbt-core`, `dbt-duckdb`

1.4 Quick start

```
astro dev start                    # starts Airflow in Docker
# open http://localhost:8080, enable the DAG "sales_pipeline" and trigger it
```

> **Note:** DuckDB allows one writer at a time. Close any DuckDB connection to the database file (for example in an SQL client) before triggering the DAG.

2. Architecture

2.1 Data flow, stage by stage

- **Source.** Five CSV files in `data/raw/`.
- **Extraction.** The task `extraction` runs `load_to_ods.py`, which copies each CSV into DuckDB as a table in the `ods` schema with `CREATE OR REPLACE TABLE`.
- **Staging.** `transformation_staging` runs `dbt run --select staging`. It creates four `stg_*` views and the `meta_data_dictionary` view.
- **Marts.** `transformation_marts` runs `dbt run --select marts`. It builds the three dimensions and the fact table as tables.
- **Quality gate.** `dbt_test` runs all 28 data tests. If one fails, the task and the DAG run fail.

2.2 Layers and schemas

DuckDB holds five schemas. The dbt schemas have the prefix `main_` because the profile uses the default schema `main` and dbt joins it with the custom schema in each model.

Layer	Schema	Objects	Type	Built by
ODS (raw)	`ods`	`raw_accounts`, `raw_data_dictionary`, `raw_products`, `raw_sales_pipeline`, `raw_sales_teams`	table	load_to_ods.py
Staging	`main_staging`	`stg_accounts`, `stg_products`, `stg_sales_pipeline`, `stg_sales_teams`	view	dbt
Meta	`main_meta`	`meta_data_dictionary`	view	dbt
Marts	`main_marts`	`dim_accounts`, `dim_products`, `dim_sales_teams`, `fact_sales_pipeline`	table	dbt
Default	`main`	empty (dbt default schema)	-	-

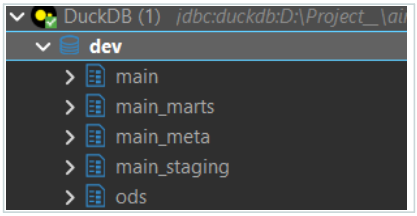


Figure 1. Schemas in the DuckDB database (DBeaver)

2.3 Design decisions

Decision	Reason
ODS landing layer	Raw CSV data is copied as is into `ods` first, so dbt always reads from the database and never from files.
Airflow orchestrates, dbt transforms	The DAG only runs commands with BashOperator. All logic is in versioned SQL models.
Views for staging, tables for marts	Staging is light cleaning and cheap to rebuild. Marts are materialised for fast queries.
Star schema	One fact and three small dimensions give simple joins and an easy model to explain.
Integer surrogate keys	Created with `ROW_NUMBER()` in each dimension; the fact table gets them by joining on the business key.
Data dictionary in `main_meta`	It is metadata, not a business entity, so it stays outside the star schema.
Separate `dev` and `prod` targets	The profile defines two DuckDB files; the DAG runs with the default target `dev`.

3. Dimensional model

3.1 Business process and grain

Business process: tracking B2B sales opportunities from prospecting to a won or lost result.

Grain: one row in `fact_sales_pipeline` is **one sales opportunity** (`opportunity_id`).

Item	Kind	Description
close_value	Measure	Revenue of the deal.
opportunity_id	Degenerate dimension	Unique identifier of the opportunity, kept in the fact.
deal_stage	Degenerate dimension	Pipeline stage: Prospecting, Engaging, Won or Lost.
engage_date	Date attribute	Date the Engaging stage started. Stored as a DATE in the fact.
close_date	Date attribute	Date the deal was Won or Lost. Stored as a DATE in the fact.

3.2 Conceptual schema

The conceptual schema shows the main entities and their relationships. Each account, product and sales agent can appear in many opportunities.

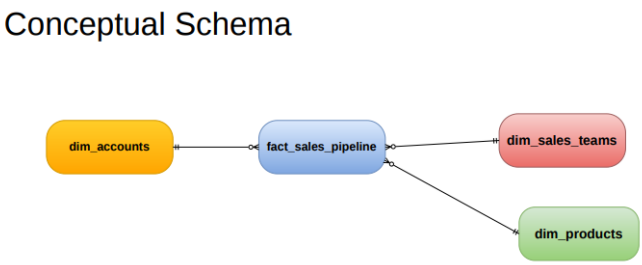


Figure 2. Conceptual schema

3.3 Logical schema

The logical schema lists the columns of each table with primary keys and foreign keys, without database-specific details.

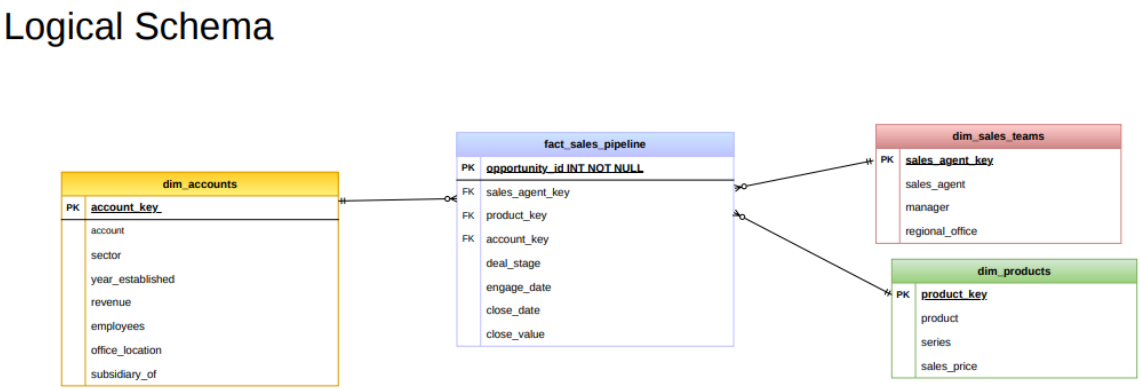


Figure 3. Logical schema

Table	Primary key	Foreign keys	Other columns
dim_accounts	account_key	-	account, sector, year_established, revenue, employees, office_location, subsidiary_of
dim_products	product_key	-	product, series, sales_price
dim_sales_teams	sales_agent_key	-	sales_agent, manager, regional_office
fact_sales_pipeline	opportunity_id	sales_agent_key, product_key, account_key	deal_stage, engage_date, close_date, close_value

3.4 Physical schema

The physical schema adds the data types and the not-null rules used in DuckDB.

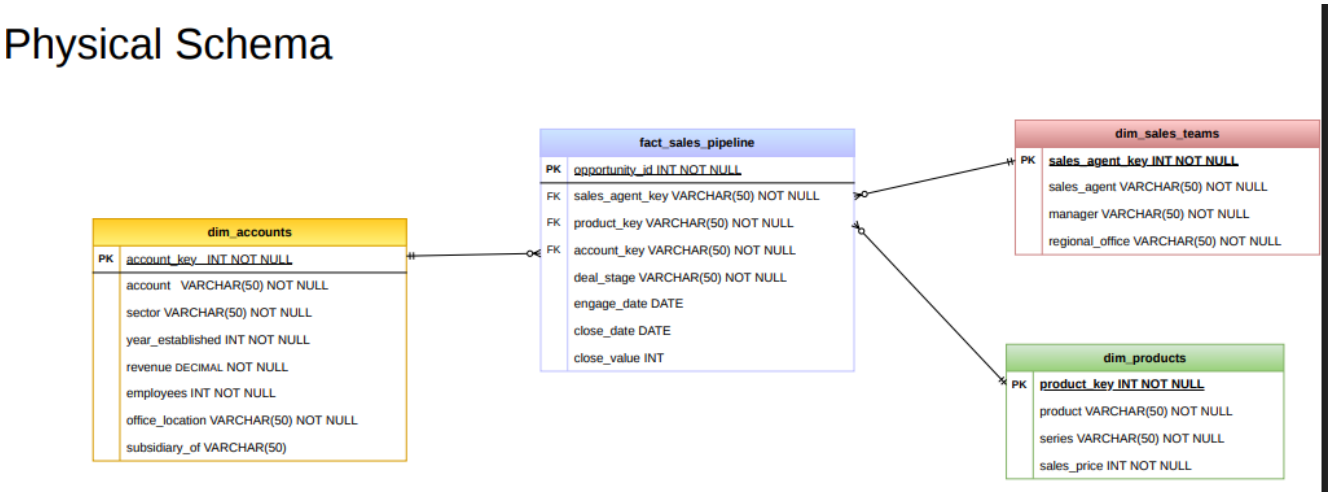


Figure 4. Physical schema

3.5 Keys

Key	Table	How it is created	Used to join on
`account_key`	dim_accounts	`ROW_NUMBER() OVER (ORDER BY account)`	account
`product_key`	dim_products	`ROW_NUMBER() OVER (ORDER BY product_name)`	product_name
`sales_agent_key`	dim_sales_teams	`ROW_NUMBER() OVER (ORDER BY sales_agent)`	sales_agent
`opportunity_id`	fact_sales_pipeline	Taken from the source file	-

Each dimension has a surrogate integer key as primary key and keeps the source name as its business key. The fact table is built with `LEFT JOIN` from the staging view to each dimension.

4. ETL implementation

4.1 Airflow DAG

The DAG `sales_pipeline` has four BashOperator tasks that run one after the other. It starts on 2026-01-01, runs `@daily`, has `catchup=False` and each task retries up to 2 times with a 1 minute delay.



Figure 5. DAG graph in Airflow: all four tasks succeeded

Task	Command	Purpose
`extraction`	`python /usr/local/airflow/scripts/load_to_ods.py`	Load the 5 CSV files into the ods schema
`transformation_staging`	`dbt run --select staging`	Build the staging and meta views
`transformation_marts`	`dbt run --select marts`	Build the dimensions and the fact
`dbt_test`	`dbt test`	Run the 28 data tests

The dbt tasks run in `/usr/local/airflow/dbt_first_project` with `--profiles-dir /usr/local/airflow/profiles`.

dags/sales_pipeline DAG file

```
from datetime import datetime, timedelta

from airflow import DAG
from airflow.operators.bash import BashOperator
from airflow.operators.python import PythonOperator
from airflow.models import Variable

default_args = {
    "owner": "mazen",
    "retries": 2,
    "retry_delay": timedelta(minutes=1),
}

with DAG(
    dag_id="sales_pipeline",
    start_date=datetime(2026, 1, 1),
    default_args=default_args,
    schedule="@daily",
    catchup=False
) as dag:

    extraction = BashOperator(
        task_id="extraction",
        bash_command="python /usr/local/airflow/scripts/load_to_ods.py",
    )

    transformation_staging = BashOperator(
        task_id="transformation_staging",
        cwd="/usr/local/airflow/dbt_first_project",
        bash_command=(
            "dbt run "
            "--profiles-dir /usr/local/airflow/profiles "
            "--select staging"
        ),
    )
```

```
    transformation_marts = BashOperator(
        task_id="transformation_marts",
        cwd="/usr/local/airflow/dbt_first_project",
        bash_command=(
            "dbt run "
            "--profiles-dir /usr/local/airflow/profiles "
            "--select marts"
        ),
    )

    test = BashOperator(
        task_id="dbt_test",
        cwd="/usr/local/airflow/dbt_first_project",
        bash_command=(
            "dbt test "
            "--profiles-dir /usr/local/airflow/profiles"
        ),
    )
    extraction >> transformation_staging >> transformation_marts >> test
```

4.2 Extraction script

`scripts/load_to_ods.py` connects to `include/dev.duckdb` and creates the `ods` schema. For each CSV file it creates a table with `CREATE OR REPLACE TABLE` and prints the row count.

Source file	ODS table	Rows loaded
accounts.csv	`ods.raw_accounts`	85
data_dictionary.csv	`ods.raw_data_dictionary`	21
products.csv	`ods.raw_products`	7
sales_pipeline.csv	`ods.raw_sales_pipeline`	8,800
sales_teams.csv	`ods.raw_sales_teams`	35

```
from pathlib import Path
import duckdb

PROJECT_ROOT = Path(__file__).resolve().parent.parent

DB_PATH = PROJECT_ROOT / "include" / "dev.duckdb"
RAW_PATH = PROJECT_ROOT / "data" / "raw"

tables = {
    "raw_accounts": RAW_PATH / "accounts.csv",
    "raw_data_dictionary": RAW_PATH / "data_dictionary.csv",
    "raw_products": RAW_PATH / "products.csv",
    "raw_sales_pipeline": RAW_PATH / "sales_pipeline.csv",
    "raw_sales_teams": RAW_PATH / "sales_teams.csv",
}

con = duckdb.connect(str(DB_PATH))
con.execute("CREATE SCHEMA IF NOT EXISTS ods")
for table_name, csv_path in tables.items():
    con.execute(f"""
        CREATE OR REPLACE TABLE ods.{table_name} AS
        SELECT *
        FROM read_csv_auto('{csv_path}', header=True)
    """)
    count = con.execute(f"SELECT COUNT(*) FROM ods.{table_name}").fetchone()[0]
    print(f"Loaded ods.{table_name}: {count} rows")

con.close()
print("ODS load complete")
```

```
▶ Log message source details
▶ Pre Execute
   7  [2026-10-09 01:05:08] INFO - Tmp dir root location: /tmp
   8  [2026-10-09 01:05:08] INFO - Running command: ['/usr/bin/bash', '-c', 'python /usr/local/airflow/scrip
   9  [2026-10-09 01:05:08] INFO - Output:
  10  [2026-10-09 01:05:09] INFO - Loaded ods.raw_accounts: 85 rows
  11  [2026-10-09 01:05:09] INFO - Loaded ods.raw_data_dictionary: 21 rows
  12  [2026-10-09 01:05:09] INFO - Loaded ods.raw_products: 7 rows
  13  [2026-10-09 01:05:09] INFO - Loaded ods.raw_sales_pipeline: 8800 rows
  14  [2026-10-09 01:05:09] INFO - Loaded ods.raw_sales_teams: 35 rows
  15  [2026-10-09 01:05:09] INFO - ODS load complete
  16  [2026-10-09 01:05:09] INFO - Command exited with return code 0
▶ Post Execute
```

Figure 6. Log of the extraction task in Airflow

4.3 dbt project

The dbt project `dbt_first_project` uses the profile `dbt_first_project`. The profile defines two DuckDB targets. Airflow uses the default target, `dev`.

profiles/profiles.yml

```yaml
dbt_first_project:
  outputs:
    dev:
      type: duckdb
      path: /usr/local/airflow/include/dev.duckdb
      threads: 1

    prod:
      type: duckdb
      path: /usr/local/airflow/include/prod.duckdb
      threads: 4

  target: dev
```

Staging models (views in main_staging and main_meta)

```sql
-- stg_accounts.sql
{{ config(materialized='view', schema='staging') }}
WITH source AS (
    SELECT
        TRIM(account) AS account,
        TRIM(sector) AS sector,
        year_established,
        revenue,
        employees,
        TRIM(office_location) AS office_location,
        subsidiary_of
    FROM {{ source('ods', 'raw_accounts') }}
)
SELECT * FROM source

-- stg_products.sql
{{ config(materialized='view', schema='staging') }}
SELECT
    product AS product_name,
    series,
    sales_price
FROM {{ source('ods', 'raw_products') }}

-- stg_sales_teams.sql
{{ config(materialized='view', schema='staging') }}
WITH source AS (
    SELECT
        TRIM(sales_agent) AS sales_agent,
        TRIM(manager) AS manager,
        TRIM(regional_office) AS regional_office
    FROM {{ source('ods', 'raw_sales_teams') }}
)
SELECT * FROM source
```

```
-- stg_sales_pipeline.sql
{{ config(materialized='view', schema='staging') }}
WITH source AS (
    SELECT
        opportunity_id,
        sales_agent,
        CASE
            WHEN product = 'GTXPro' THEN 'GTX Pro'     -- data quality fix: product name
            ELSE product
        END AS product_name,
        account,
        deal_stage,
        engage_date,
        close_date,
        close_value
    FROM {{ source('ods', 'raw_sales_pipeline') }}
)
SELECT * FROM source
-- meta_data_dictionary.sql
{{ config(materialized='view', schema='meta') }}
WITH sorces AS (
    SELECT
        "Table"       AS table_name,
        "Field"       AS field_name,
        "Description" AS description_name
    FROM {{ source('ods', 'raw_data_dictionary') }}
)
SELECT * FROM sorces
```

Mart models (tables in main_marts)

```
-- dim_accounts.sql
{{ config(materialized='table', schema='marts') }}
WITH accounts AS (
    SELECT
        ROW_NUMBER() OVER (ORDER BY account) AS account_key,
        account, sector, year_established, revenue, employees, office_location, subsidiary_of
    FROM {{ ref('stg_accounts') }}
)
SELECT * FROM accounts

-- dim_products.sql
{{ config(materialized='table', schema='marts') }}
SELECT
    ROW_NUMBER() OVER (ORDER BY product_name) AS product_key,
    product_name, series, sales_price
FROM {{ ref('stg_products') }}

-- dim_sales_teams.sql
{{ config(materialized='table', schema='marts') }}
SELECT
    ROW_NUMBER() OVER (ORDER BY sales_agent) AS sales_agent_key,
    sales_agent, manager, regional_office
FROM {{ ref('stg_sales_teams') }}
```

```
-- fact_sales_pipeline.sql
{{ config(materialized='table', schema='marts') }}
SELECT
    sp.opportunity_id,
    st.sales_agent_key,
    dp.product_key,
    sa.account_key,
    sp.deal_stage,
    sp.engage_date,
    sp.close_date,
    sp.close_value
FROM {{ ref('stg_sales_pipeline') }} AS sp
    LEFT JOIN {{ ref('dim_sales_teams') }} AS st ON sp.sales_agent = st.sales_agent
    LEFT JOIN {{ ref('dim_products') }}    AS dp ON sp.product_name = dp.product_name
    LEFT JOIN {{ ref('dim_accounts') }}    AS sa ON sp.account = sa.account
```

4.4 Run results and tests

The Airflow logs below come from the run of 2026-10-09. dbt 1.12.5 with the DuckDB adapter 1.11.0 found 9 models, 28 data tests and 5 sources.

Task	Result	Time
transformation_staging	5 view models created (4 staging + meta dictionary)	0.85 s
transformation_marts	4 table models created (3 dimensions + fact)	0.36 s
dbt_test	28 tests: PASS=28 WARN=0 ERROR=0	0.87 s

```
[2026-10-09 01:05:22] INFO - 22:05:22  1 of 5 START sql view model main_meta.meta_data_dictionary ..... [RUN]
[2026-10-09 01:05:22] INFO - 22:05:22  1 of 5 OK created sql view model main_meta.meta_data_dictionary ..... [OK in 0.11s]
[2026-10-09 01:05:22] INFO - 22:05:22  2 of 5 START sql view model main_staging.stg_accounts ..... [RUN]
[2026-10-09 01:05:22] INFO - 22:05:22  2 of 5 OK created sql view model main_staging.stg_accounts ..... [OK in 0.05s]
[2026-10-09 01:05:22] INFO - 22:05:22  3 of 5 START sql view model main_staging.stg_products ..... [RUN]
[2026-10-09 01:05:22] INFO - 22:05:22  3 of 5 OK created sql view model main_staging.stg_products ..... [OK in 0.04s]
[2026-10-09 01:05:22] INFO - 22:05:22  4 of 5 START sql view model main_staging.stg_sales_pipeline ..... [RUN]
[2026-10-09 01:05:22] INFO - 22:05:22  4 of 5 OK created sql view model main_staging.stg_sales_pipeline ..... [OK in 0.04s]
[2026-10-09 01:05:22] INFO - 22:05:22  5 of 5 START sql view model main_staging.stg_sales_teams ..... [RUN]
[2026-10-09 01:05:22] INFO - 22:05:22  5 of 5 OK created sql view model main_staging.stg_sales_teams ..... [OK in 0.05s]
[2026-10-09 01:05:22] INFO - 22:05:22
[2026-10-09 01:05:22] INFO - 22:05:22  Finished running 5 view models in 0 hours 0 minutes and 0.85 seconds (0.85s).
[2026-10-09 01:05:22] INFO - 22:05:22
[2026-10-09 01:05:22] INFO - 22:05:22  Completed successfully
[2026-10-09 01:05:22] INFO - 22:05:22
[2026-10-09 01:05:22] INFO - 22:05:22  Done. PASS=5 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=5
[2026-10-09 01:05:24] INFO - Command exited with return code 0
Post Execute
```

Figure 7. Log of transformation_staging: 5 views created

```
[2026-10-09 01:05:25] INFO - Running command: ['/usr/bin/bash', '-c', 'dbt run --profiles-dir /usr/local/airflow/profiles --select marts']
[2026-10-09 01:05:25] INFO - Output:
[2026-10-09 01:05:29] INFO - 22:05:29  Running with dbt=1.12.5
[2026-10-09 01:05:29] INFO - 22:05:29  Registered adapter: duckdb=1.11.0
[2026-10-09 01:05:29] INFO - 22:05:29  [WARNING]: Configuration paths exist in your dbt_project.yml file which do not apply to any resource
[2026-10-09 01:05:29] INFO - There are 1 unused configuration paths:
[2026-10-09 01:05:29] INFO - - models.dbt_first_project.example
[2026-10-09 01:05:29] INFO - 22:05:29  Found 9 models, 28 data tests, 5 sources, 500 macros
[2026-10-09 01:05:29] INFO - 22:05:29
[2026-10-09 01:05:29] INFO - 22:05:29  Concurrency: 1 threads (target='dev')
[2026-10-09 01:05:29] INFO - 22:05:29
[2026-10-09 01:05:30] INFO - 22:05:30  1 of 4 START sql table model main_marts.dim_accounts ..... [RUN]
[2026-10-09 01:05:30] INFO - 22:05:30  1 of 4 OK created sql table model main_marts.dim_accounts ..... [OK in 0.10s]
[2026-10-09 01:05:30] INFO - 22:05:30  2 of 4 START sql table model main_marts.dim_products ..... [RUN]
[2026-10-09 01:05:30] INFO - 22:05:30  2 of 4 OK created sql table model main_marts.dim_products ..... [OK in 0.04s]
[2026-10-09 01:05:30] INFO - 22:05:30  3 of 4 START sql table model main_marts.dim_sales_teams ..... [RUN]
[2026-10-09 01:05:30] INFO - 22:05:30  3 of 4 OK created sql table model main_marts.dim_sales_teams ..... [OK in 0.03s]
[2026-10-09 01:05:30] INFO - 22:05:30  4 of 4 START sql table model main_marts.fact_sales_pipeline ..... [RUN]
[2026-10-09 01:05:30] INFO - 22:05:30  4 of 4 OK created sql table model main_marts.fact_sales_pipeline ..... [OK in 0.05s]
[2026-10-09 01:05:30] INFO - 22:05:30
[2026-10-09 01:05:30] INFO - 22:05:30  Finished running 4 table models in 0 hours 0 minutes and 0.36 seconds (0.36s).
[2026-10-09 01:05:30] INFO - 22:05:30
[2026-10-09 01:05:30] INFO - 22:05:30  Completed successfully
[2026-10-09 01:05:30] INFO - 22:05:30
[2026-10-09 01:05:30] INFO - 22:05:30  Done. PASS=4 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=4
[2026-10-09 01:05:31] INFO - Command exited with return code 0
Post Execute
```

Figure 8. Log of transformation_marts: 4 tables created

```
[2026-10-09 01:05:38] INFO - 22:05:38  24 of 28 START test not_null_stg_sales_teams_sales_agent ..... [RUN]
[2026-10-09 01:05:38] INFO - 22:05:38  24 of 28 PASS not_null_stg_sales_teams_sales_agent ..... [PASS in 0.02s]
[2026-10-09 01:05:38] INFO - 22:05:38  25 of 28 START test unique_dim_accounts_account_key ..... [RUN]
[2026-10-09 01:05:38] INFO - 22:05:38  25 of 28 PASS unique_dim_accounts_account_key ..... [PASS in 0.02s]
[2026-10-09 01:05:38] INFO - 22:05:38  26 of 28 START test unique_dim_products_product_key ..... [RUN]
[2026-10-09 01:05:38] INFO - 22:05:38  26 of 28 PASS unique_dim_products_product_key ..... [PASS in 0.02s]
[2026-10-09 01:05:38] INFO - 22:05:38  27 of 28 START test unique_dim_sales_teams_sales_agent_key ..... [RUN]
[2026-10-09 01:05:38] INFO - 22:05:38  27 of 28 PASS unique_dim_sales_teams_sales_agent_key ..... [PASS in 0.02s]
[2026-10-09 01:05:38] INFO - 22:05:38  28 of 28 START test unique_stg_accounts_account ..... [RUN]
[2026-10-09 01:05:38] INFO - 22:05:38  28 of 28 PASS unique_stg_accounts_account ..... [PASS in 0.01s]
[2026-10-09 01:05:38] INFO - 22:05:38
[2026-10-09 01:05:38] INFO - 22:05:38  Finished running 28 data tests in 0 hours 0 minutes and 0.87 seconds (0.87s).
[2026-10-09 01:05:38] INFO - 22:05:38
[2026-10-09 01:05:38] INFO - 22:05:38  Completed successfully
[2026-10-09 01:05:38] INFO - 22:05:38
[2026-10-09 01:05:38] INFO - 22:05:38  Done. PASS=28 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=28
[2026-10-09 01:05:39] INFO - Command exited with return code 0
Post Execute
```

Figure 9. Log of dbt_test: 28 of 28 tests pass

Tests in the project

Test	Applied to (examples from the log)
not_null	stg_accounts: sector, year_established; stg_products: product_name, sales_price, series; stg_sales_pipeline: opportunity_id, deal_stage, product_name, sales_agent; stg_sales_teams: sales_agent, manager, regional_office
unique	dim_accounts.account_key, dim_products.product_key, dim_sales_teams.sales_agent_key, stg_accounts.account

5. Setup and operations

5.1 Prerequisites

- Docker and the Astro CLI.
- The five CSV files in `data/raw/`.
- Python packages from `requirements.txt`: `astronomer-cosmos`, `dbt-core`, `dbt-duckdb`.

5.2 Run the pipeline

```
astro dev start        # build the image and start Airflow (UI on http://localhost:8080)
astro dev stop         # stop the containers
```

- Open the Airflow UI, enable the DAG `sales_pipeline` and trigger it, or wait for the daily schedule.
- Check that the four tasks `extraction`, `transformation_staging`, `transformation_marts` and `dbt_test` are green.

5.3 Verify the result

```
-- row counts of the warehouse
SELECT count(*) FROM main_marts.fact_sales_pipeline;     -- should equal ods.raw_sales_pipeline (8800)

-- fact rows without a matching dimension row
SELECT
    count(*) FILTER (WHERE account_key     IS NULL) AS no_account,
    count(*) FILTER (WHERE product_key     IS NULL) AS no_product,
    count(*) FILTER (WHERE sales_agent_key IS NULL) AS no_agent
FROM main_marts.fact_sales_pipeline;
```

5.4 Troubleshooting

Symptom	Likely cause	Fix
Task fails with a DuckDB lock error	Another program has the database file open	Close the SQL client or notebook, then clear the task in Airflow
Schemas are named `main_marts`	dbt joins the default schema `main` with the custom one	Expected behaviour here; query the schemas with the `main_` prefix
dbt warns about `tests/acount_text.yml`	The file points to a node `accounts` that does not exist	Point it to `stg_accounts` or move its tests to `schema.yml`
Fact rows have NULL keys	Name in the pipeline does not match the dimension (for example a trailing space)	Apply `TRIM` in `stg_sales_pipeline` and re-run

6. Limitations and next steps

- **Keys depend on sort order.** Surrogate keys come from `ROW_NUMBER()`. Because all tables are rebuilt together in every run, the fact and dimensions stay consistent, but the same account can get a different key after the data changes.
- **Joins on names.** The fact joins on `sales_agent`, `product_name` and `account`. `TRIM` is not trimmed in `stg_sales_pipeline`, so a stray space would produce a NULL key.
- **No date dimension.** `engage_date` and `close_date` stay as DATE columns in the fact. A `dim_date` would add year, quarter and month attributes.
- **No history.** Dimensions are overwritten on each run (SCD Type 1). dbt snapshots can track changes.
- **Full rebuild.** Every run reloads all CSV files and rebuilds all tables, which is fine for 8,800 rows.
- **Foreign-key checks.** Add `relationships` tests between the fact keys and the dimension keys to catch orphan rows.
- **Small clean-ups.** The `example` config in `dbt_project.yml` matches no models (dbt warning), the DAG imports `PythonOperator` and `Variable` without using them, and `astronomer-cosmos` is installed but not used.
- **Production target.** The `prod` target exists in the profile, but the DAG runs on `dev`.

Appendix A. Data dictionary

The source data dictionary (21 rows, loaded into `ods.raw_data_dictionary` and exposed as `main_meta.meta_data_dictionary`) with the column each field becomes in the warehouse. It is metadata, so it is not part of the star schema.

Table	Field	Description	Maps to
accounts	account	Company name	dim_accounts.account
accounts	sector	Industry	dim_accounts.sector
accounts	year_established	Year established	dim_accounts.year_established
accounts	revenue	Annual revenue (millions of USD)	dim_accounts.revenue
accounts	employees	Number of employees	dim_accounts.employees
accounts	office_location	Headquarters	dim_accounts.office_location
accounts	subsidiary_of	Parent company	dim_accounts.subsidiary_of
products	product	Product name	dim_products.product_name
products	series	Product series	dim_products.series
products	sales_price	Suggested retail price	dim_products.sales_price
sales_teams	sales_agent	Sales agent	dim_sales_teams.sales_agent
sales_teams	manager	Respective sales manager	dim_sales_teams.manager
sales_teams	regional_office	Regional office	dim_sales_teams.regional_office
sales_pipeline	opportunity_id	Unique identifier	fact.opportunity_id (PK)
sales_pipeline	sales_agent	Sales agent	fact.sales_agent_key (FK)
sales_pipeline	product	Product name	fact.product_key (FK)
sales_pipeline	account	Company name	fact.account_key (FK)
sales_pipeline	deal_stage	Pipeline stage (Prospecting > Engaging > Won / Lost)	fact.deal_stage
sales_pipeline	engage_date	Date the Engaging stage was initiated	fact.engage_date
sales_pipeline	close_date	Date the deal was Won or Lost	fact.close_date
sales_pipeline	close_value	Revenue from the deal	fact.close_value (measure)